

## Week 8 - Graded Mini Project

### Learning Outcome Addressed

- Data Extraction and Manipulation Using SQL and Pandas
- Deriving Business Insights through Exploratory Data Analysis and Visualisation

### Objective

This mini project is designed to help you explore and analyse retail sales data using SQL, Pandas, and Python visualisations. You'll work with two datasets: one containing stock information and another recording sales transactions. Through a combination of structured queries and Python-based EDA, you'll derive business insights such as bestselling items, customer purchasing patterns, and revenue trends.

### Submission Instructions

Please document your response on the following pages.

Once you have completed the activity, save the file as a PDF and upload it. Be sure to name the file as **Module 8: Graded Mini Project\_[Your last name]**.

Your submission will be considered complete when it meets the following criteria:

- Includes all the key elements outlined in the activity instructions and the rubric.
- Add screenshots of visual outputs wherever required.
- Adheres to the submission guidelines.
- Is submitted on time.

***This is a required activity and counts towards programme completion.***

Reflect on the task and respond to the following questions.

#### **A. Basic SQL Queries (5 Tasks)- Load csv files in database. (Note: Kindly provide the solution in the form of an SQL query.)**

Question: Retrieve all stock items that contain the word "T-LIGHT" in their description.

```
select * from stockdetails where description like '%T-LIGHT%';
```

Question: Calculate the **total quantity sold per StockCode**.

```
select StockCode, sum(Quantity) as sum_quantity from sales group by StockCode order by StockCode;
```

Question: Find **total revenue** ( $\text{Quantity} \times \text{UnitPrice}$ ) per **CustomerID**.

```
select CustomerID, sum((Quantity * UnitPrice)) as total_revenue from sales  
group by CustomerID order by CustomerID;
```

Question: Get a list of all **invoices** and count of **distinct stock items** per invoice.

```
SELECT  
    InvoiceNo,  
    COUNT(DISTINCT StockCode) AS distinct_item_count  
FROM sales  
GROUP BY InvoiceNo  
ORDER BY InvoiceNo;
```

Question: Perform an **inner join** between sales and stock details to display full item names along with **total revenue per item**.

```
SELECT  
    sd.Description, sd.StockCode,  
    SUM(s.quantity * s.unitprice) AS total_revenue  
FROM sales s INNER JOIN stockdetails sd ON s.StockCode = sd.StockCode  
GROUP BY sd.Description, sd.StockCode  
ORDER BY total_revenue desc ;
```

**B. EDA Using Pandas (5 Tasks) (Note: Kindly provide the lines of python code.)**

Question: Check for **missing values**, **data types**, and **duplicates** in both datasets.

```
import pandas as pd

pd.set_option('display.max_columns', None)

pd.set_option('display.width', None)

pd.set_option('display.max_rows', None)


# 1) Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")


# --- Step 1: Check for missing values, data types, and duplicates ---

print("StockDetails Info:")

print(stock_df.info())

print("\nMissing values in StockDetails:\n", stock_df.isnull().sum())

print("\nDuplicates in StockDetails:", stock_df.duplicated().sum())


print("\nSales Info:")

print(sales_df.info())

print("\nMissing values in Sales:\n", sales_df.isnull().sum())

print("\nDuplicates in Sales:", sales_df.duplicated().sum())
```

Question: Convert InvoiceDate into datetime, and extract:

- Invoice date
- Month
- Hour of transaction

```
import pandas as pd

pd.set_option('display.max_columns', None)

pd.set_option('display.width', None)

pd.set_option('display.max_rows', None)


# 1) Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")


# Convert InvoiceDate to datetime and extract Invoice date, Month, Hour
sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')

# Invoice date (date only)
sales_df['Invoice_date'] = sales_df['InvoiceDate'].dt.date

# Month number (1-12)
sales_df['Month'] = sales_df['InvoiceDate'].dt.month

# Hour of transaction (0-23)
sales_df['Hour'] = sales_df['InvoiceDate'].dt.hour

print('Converted InvoiceDate — NaT values:', sales_df['InvoiceDate'].isna().sum())

# Show sample of new columns
sales_df[['InvoiceDate', 'Invoice_date', 'Month', 'Hour']].head()
```

Question: Add a new column  $TotalPrice = Quantity \times UnitPrice$ .

```
import pandas as pd

pd.set_option('display.max_columns', None)
```

```
pd.set_option('display.width', None)

pd.set_option('display.max_rows', None)

# 1) Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# --- Step 5: Add new column TotalPrice = Quantity × UnitPrice ---
sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']

print(sales_df['TotalPrice'])
```

Question: Identify the **top 3 bestselling items** by quantity sold.

```
import pandas as pd

pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_rows', None)

# Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# --- Step 6: Identify top 3 bestselling items by quantity sold ---
top_items = (
    sales_df.groupby('StockCode')['Quantity']
    .sum()
```

```
.reset_index()

.sort_values(by='Quantity', ascending=False)

.head(3)

)

print("\nTop 3 Bestselling Items by Quantity Sold:")

print(top_items)
```

Question: Find out how many **unique customers** made purchases and the **average quantity per invoice**.

```
import pandas as pd

pd.set_option('display.max_columns', None)

pd.set_option('display.width', None)

pd.set_option('display.max_rows', None)

# Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")

sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# --- Step 7: Unique customers and average quantity per invoice ---

unique_customers = sales_df['CustomerID'].nunique()

avg_quantity_per_invoice = (

    sales_df.groupby('InvoiceNo')['Quantity'].sum().mean()

)
```

**C. Visualisation (5 Tasks)** (Note: Kindly provide the lines of python code and the screenshots of charts and plots.)

Question: Plot a **bar chart** of top 10 items by **quantity sold**.

```
import pandas as pd

import matplotlib.pyplot as plt

# Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")

sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# Ensure InvoiceDate is datetime

sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')

# Add TotalPrice column

sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']

# -----

# 11. Bar chart of top 10 items by quantity sold

top10_items = (

    sales_df.groupby('StockCode')['Quantity']

    .sum()

    .reset_index()

    .merge(stock_df, on='StockCode', how='left')

    .sort_values(by='Quantity', ascending=False)

    .head(10)

)

# figure size

plt.figure(figsize=(10,6))
```

#bar chart with X & Y co-ordinates

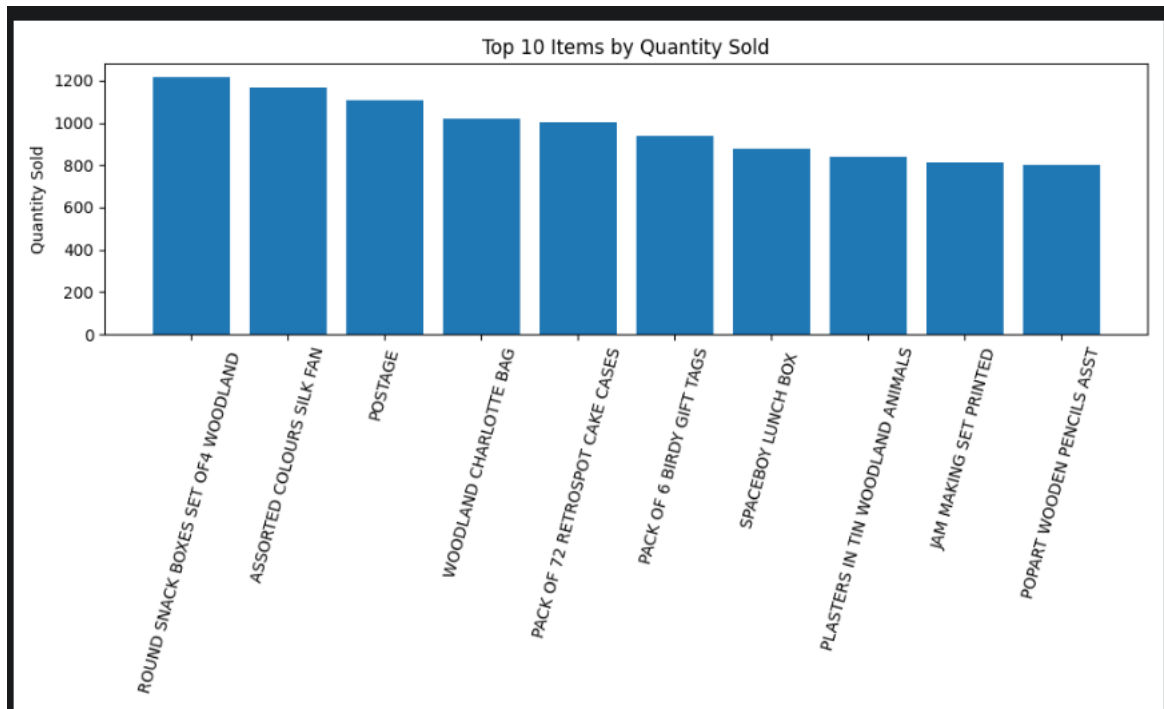
```
plt.bar(top10_items['Description'], top10_items['Quantity'])
```

```
plt.xticks(rotation=75)
```

```
plt.title('Top 10 Items by Quantity Sold')
```

```
plt.tight_layout()
```

```
plt.show()
```



Question: Plot **total sales per hour of the day** to understand peak shopping hours.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Load datasets
```

```
stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
```

```
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")
```



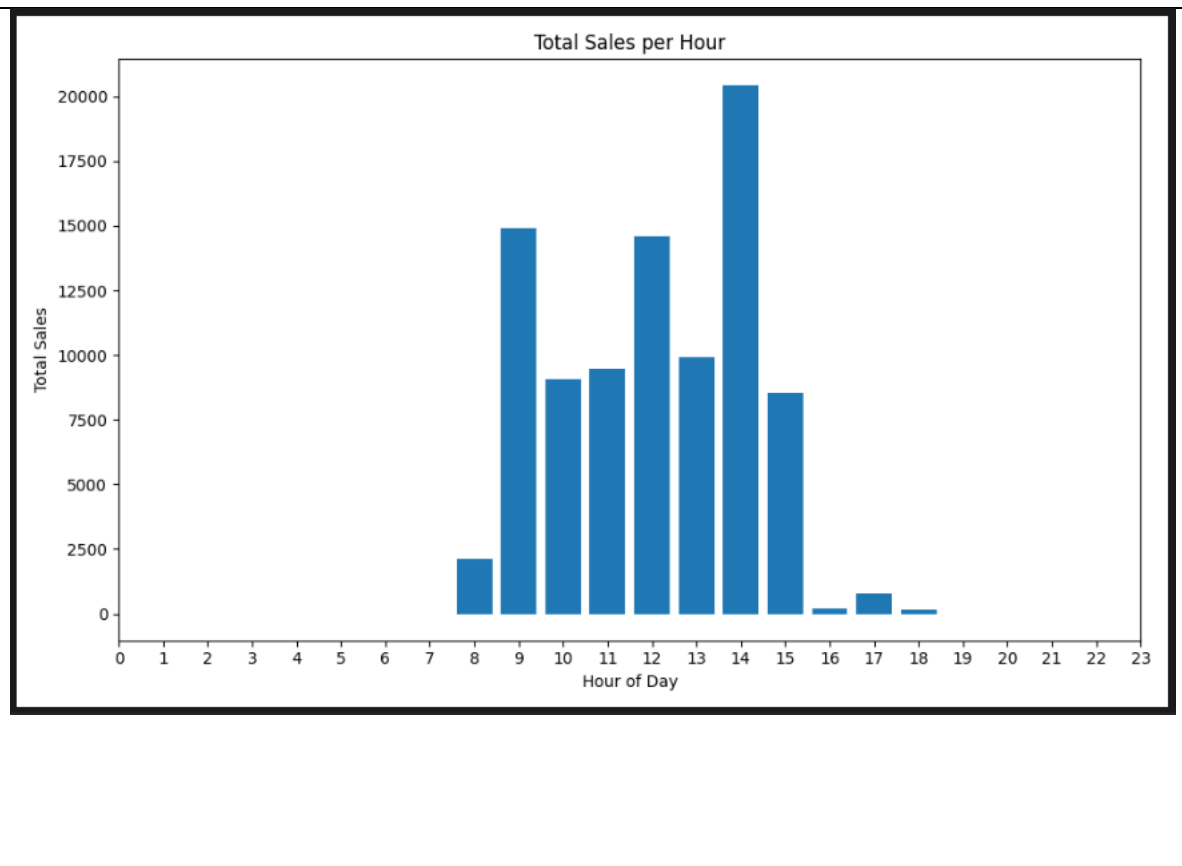
```
# Ensure InvoiceDate is datetime
sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')

# Add TotalPrice column
sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']

# -----

# 12. Total sales per hour of the day
sales_df['Hour'] = sales_df['InvoiceDate'].dt.hour
sales_per_hour = sales_df.groupby('Hour')['TotalPrice'].sum().reset_index()

plt.figure(figsize=(10,6))
plt.bar(sales_per_hour['Hour'], sales_per_hour['TotalPrice'])
plt.title('Total Sales per Hour')
plt.xlabel('Hour of Day')
plt.ylabel('Total Sales')
plt.xticks(range(0,24))
plt.tight_layout()
plt.show()
```



Question: Create a **pie chart** showing revenue distribution among top 5 customers.

```
import pandas as pd

import matplotlib.pyplot as plt

# Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# Ensure InvoiceDate is datetime
sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')
```

```
# Add TotalPrice column

sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']

# -----

# 13. Pie chart of revenue distribution among top 5 customers

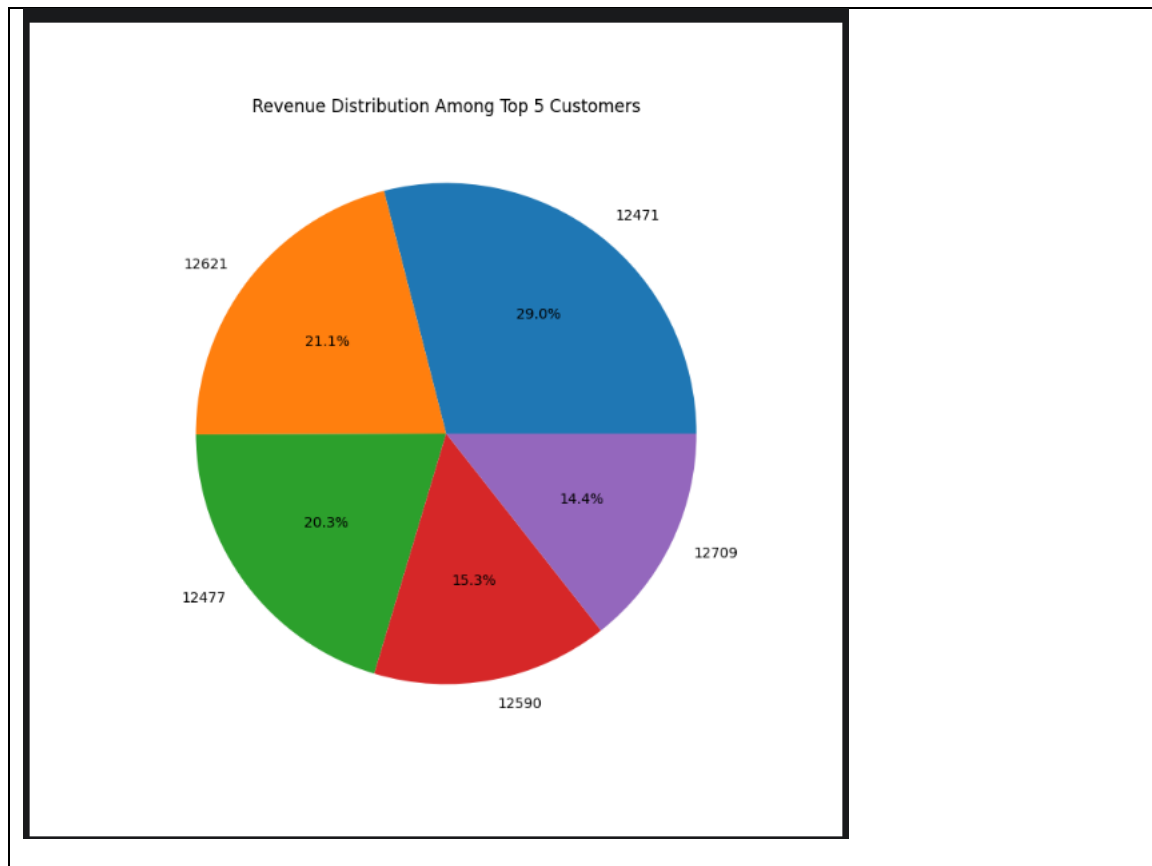
top_customers = (
    sales_df.groupby('CustomerID')['TotalPrice']
    .sum()
    .reset_index()
    .sort_values(by='TotalPrice', ascending=False)
    .head(5)
)

plt.figure(figsize=(8,8))

plt.pie(top_customers['TotalPrice'], labels=top_customers['CustomerID'], autopct='%1.1f%%')

plt.title('Revenue Distribution Among Top 5 Customers')

plt.show()
```



Question: Visualise the **monthly revenue trend** using a line chart.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load datasets
stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# Ensure InvoiceDate is datetime
sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')

# Add TotalPrice column
sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']
```

# 14. Monthly revenue trend (line chart)

# Extracting only Month from Date

```
sales_df['Month'] = sales_df['InvoiceDate'].dt.to_period('M')
```

```
monthly_revenue = sales_df.groupby('Month')['TotalPrice'].sum().reset_index()
```

```
plt.figure(figsize=(10,6))
```

```
plt.plot(monthly_revenue['Month'].astype(str), monthly_revenue['TotalPrice'], marker='o')
```

```
plt.title('Monthly Revenue Trend')
```

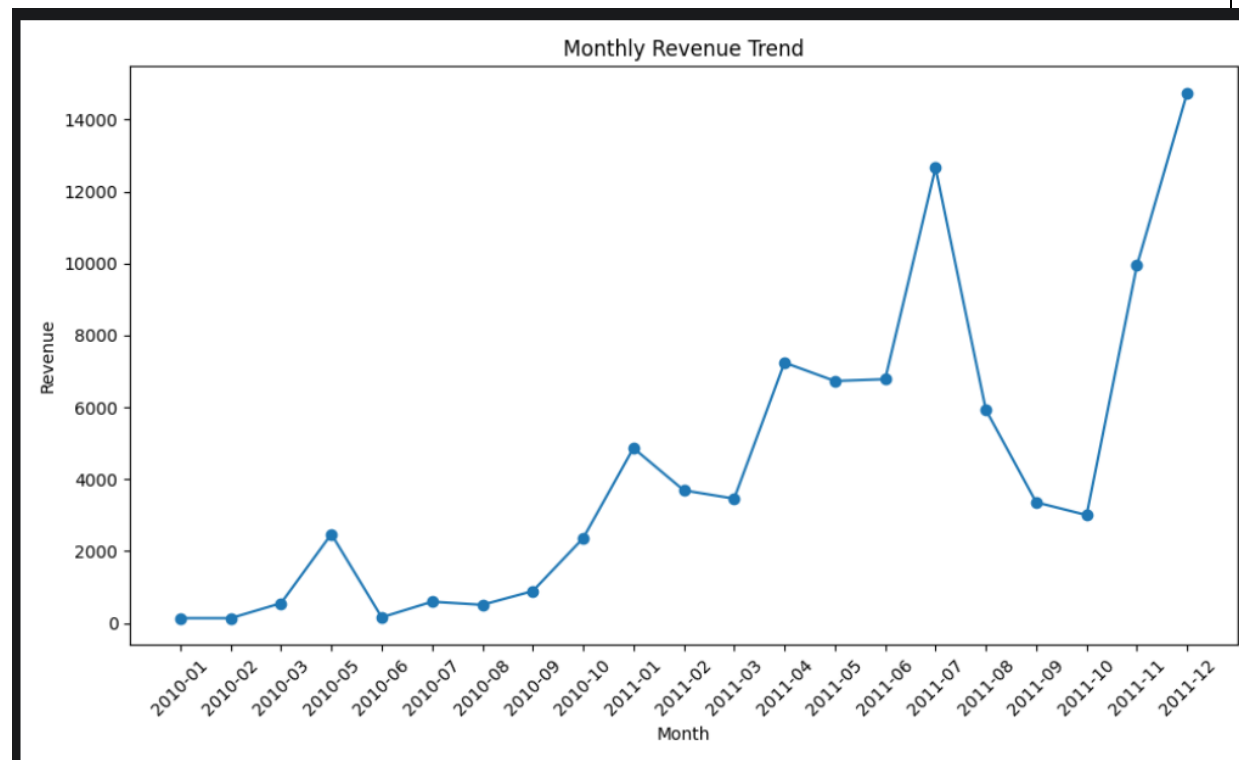
```
plt.xlabel('Month')
```

```
plt.ylabel('Revenue')
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```



Question: Create a **stacked bar chart** of top 5 invoices by revenue, showing contribution from each item.

```
import pandas as pd

import matplotlib.pyplot as plt

# Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# Ensure InvoiceDate is datetime

sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')

# Add TotalPrice column

sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']
# -----

# 15. Stacked bar chart of top 5 invoices by revenue

invoice_revenue = sales_df.groupby('InvoiceNo')['TotalPrice'].sum().reset_index()
top5_invoices = invoice_revenue.sort_values(by='TotalPrice', ascending=False).head(5)

invoice_items = sales_df[sales_df['InvoiceNo'].isin(top5_invoices['InvoiceNo'])]

stacked_data =
invoice_items.groupby(['InvoiceNo', 'Description'])['TotalPrice'].sum().unstack(fill_value=0)

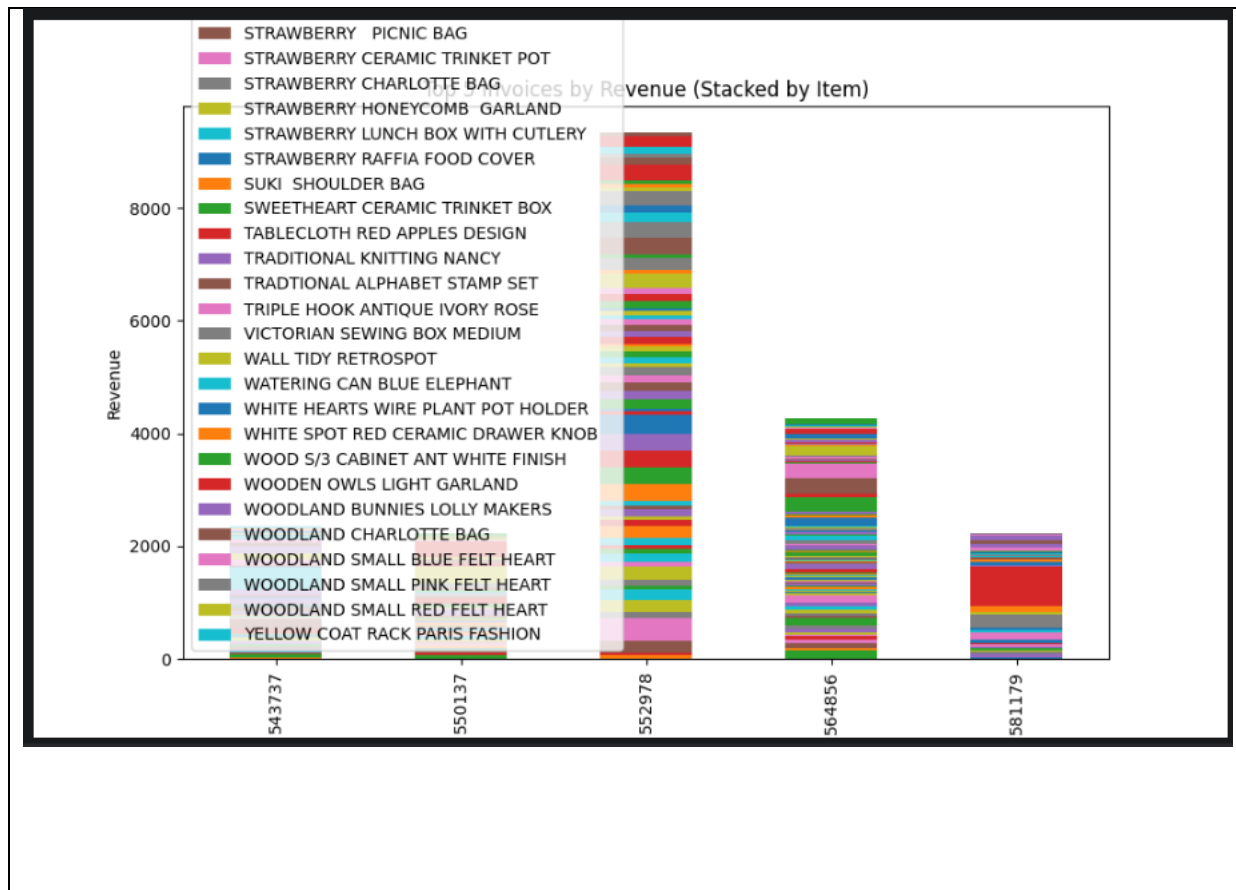
stacked_data.plot(kind='bar', stacked=True, figsize=(10,6))

plt.title('Top 5 Invoices by Revenue (Stacked by Item)')

plt.ylabel('Revenue')

plt.tight_layout()

plt.show()
```



#### D. Business Insights (5 Tasks) (Note: Kindly provide the lines of python code.)

Question: Identify which **product** generates the highest total revenue.

```
import pandas as pd

# Load datasets
stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# Ensure InvoiceDate is datetime
sales_df["InvoiceDate"] = pd.to_datetime(sales_df["InvoiceDate"], errors='coerce')

# Add TotalPrice column
```

```

sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']

# -----

# 16. Identify which product generates the highest total revenue

product_revenue = (
    sales_df.groupby('StockCode')['TotalPrice']
    .sum()
    .reset_index()
    .merge(stock_df, on='StockCode', how='left')
    .sort_values(by='TotalPrice', ascending=False)
)

highest_revenue_product = product_revenue.iloc[0]

print("Product with Highest Revenue:")

print(highest_revenue_product[['StockCode', 'Description', 'TotalPrice']])

```

Question: Determine **average order value per invoice**.

```

import pandas as pd

# Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")

sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# Ensure InvoiceDate is datetime

sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')

```



```
# Add TotalPrice column

sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']

# -----

# 17. Determine average order value per invoice

invoice_totals = sales_df.groupby('InvoiceNo')['TotalPrice'].sum()

avg_order_value = invoice_totals.mean()

print("\nAverage Order Value per Invoice:", avg_order_value)
```

Question: Find the **customer with the highest number of transactions**.

```
import pandas as pd

# Load datasets

stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# Ensure InvoiceDate is datetime

sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')

# Add TotalPrice column

sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']

# 18. Find the customer with the highest number of transactions

customer_transactions = sales_df.groupby('CustomerID')['InvoiceNo'].nunique().reset_index()

top_customer = customer_transactions.sort_values(by='InvoiceNo', ascending=False).iloc[0]

print("\nCustomer with Highest Number of Transactions:")

print(top_customer)
```

Question: Check how many products in the sales data do **not have a matching entry** in the stockDetails file.

```
import pandas as pd

# Load datasets
stock_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\StockDetails.csv")
sales_df = pd.read_csv(r"C:\Users\LENOVO\Downloads\sales.csv")

# Ensure InvoiceDate is datetime
sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'], errors='coerce')

# Add TotalPrice column
sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['UnitPrice']

# -----

# 19. Check how many products in sales data do not have a matching entry in stockDetails
missing_products = sales_df[~sales_df['StockCode'].isin(stock_df['StockCode'])]

num_missing_products = missing_products['StockCode'].nunique()

print("\nNumber of Products in Sales without Matching StockDetails:",
      num_missing_products)
```